



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	Dr. Mohammed Razia Alangir Banu
Student Name	Rayanki Himavarshini
Roll Number	2520030141
Branch	CSE
Course Outcome	CO4

Case Study Topic: AgroChain – Smart Agriculture Supply Chain Management System.

Work Done Summary:

In this case study, we implemented and analyzed Dijkstra's Algorithm, Bellman-Ford Algorithm, and Floyd-Warshall Algorithm for finding shortest paths in transportation, logistics, and routing systems.

The algorithms were tested on different graph types, including weighted and unweighted graphs, directed and undirected graphs, and graphs containing negative-weight edges. Their correctness, efficiency, and applicability were compared under various routing scenarios.

The implementation was developed in Java to efficiently calculate optimal routes, minimize transportation costs, and improve delivery planning in logistics networks.

Implementation Details:

- Created graph data structures using adjacency lists and matrices to represent routing networks.
- Implemented **Dijkstra's Algorithm** for finding shortest paths in graphs with non-negative edge weights.
- Implemented **Bellman-Ford Algorithm** to handle graphs containing negative-weight edges and detect negative cycles.
- Implemented **Floyd-Warshall Algorithm** to compute shortest paths between all pairs of vertices.

- Tested the algorithms on directed and undirected weighted graphs.
- Verified shortest-path results for different source and destination nodes.
- Compared execution time and memory usage of the three algorithms.
- Analyzed algorithm performance in logistics and transportation routing case studies.
- Evaluated the suitability of each algorithm based on graph characteristics and network requirements.
- Documented observations regarding correctness, efficiency, and practical applications in real-world routing systems.

Input Screenshot :

```
import java.util.*;

public class ShortestPathAlgorithms {

    static final int INF = 99999;

    // Dijkstra's Algorithm
    static void dijkstra(int graph[][], int src) {
        int V = graph.length;
        int dist[] = new int[V];
        boolean visited[] = new boolean[V];

        Arrays.fill(dist, INF);
        dist[src] = 0;

        for (int count = 0; count < V - 1; count++) {
            int u = -1;
            int min = INF;

            for (int i = 0; i < V; i++) {
                if (!visited[i] && dist[i] < min) {
                    min = dist[i];
                    u = i;
                }
            }

            visited[u] = true;
```

```

        for (int v = 0; v < V; v++) {
            if (!visited[v] && graph[u][v] != 0 &&
                dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    System.out.println("Dijkstra's Algorithm:");
    for (int i = 0; i < V; i++)
        System.out.println("Distance from " + src + " to " + i + " = " + dist[i]);
}

// Bellman-Ford Algorithm
static void bellmanFord(int edges[][], int V, int E, int src) {
    int dist[] = new int[V];
    Arrays.fill(dist, INF);
    dist[src] = 0;

    for (int i = 1; i < V; i++) {
        for (int j = 0; j < E; j++) {
            int u = edges[j][0];
            int v = edges[j][1];
            int w = edges[j][2];

            if (dist[u] != INF && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
            }
        }
    }
}

```

```

    System.out.println("\nBellman-Ford Algorithm:");
    for (int i = 0; i < V; i++)
        System.out.println("Distance from " + src + " to " + i + " = " + dist[i]);
}

// Floyd-Warshall Algorithm
static void floydWarshall(int graph[][][]) {
    int V = graph.length;
    int dist[][] = new int[V][V];

    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
            dist[i][j] = graph[i][j];

    for (int k = 0; k < V; k++) {
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                if (dist[i][k] != INF && dist[k][j] != INF
                    && dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }
}

```

```

        System.out.println("\nFloyd-Warshall Algorithm:");
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++)
                System.out.print(dist[i][j] + "\t");
            System.out.println();
        }
    }

    public static void main(String[] args) {

        int graph[][] = {
            {0, 4, 2, 0},
            {4, 0, 1, 5},
            {2, 1, 0, 8},
            {0, 5, 8, 0}
        };

        dijkstra(graph, 0);

        int edges[][] = {
            {0, 1, 4},
            {0, 2, 2},
            {1, 2, 1},
            {1, 3, 5},
            {2, 3, 8}
        };
    }

```

```

        bellmanFord(edges, 4, 5, 0);

        int fwGraph[][] = {
            {0, 4, 2, INF},
            {4, 0, 1, 5},
            {2, 1, 0, 8},
            {INF, 5, 8, 0}
        };

        floydWarshall(fwGraph);
    }
}

```

Output Screenshot :

```
Dijkstra's Algorithm:
Distance from 0 to 0 = 0
Distance from 0 to 1 = 3
Distance from 0 to 2 = 2
Distance from 0 to 3 = 8
```

```
Bellman-Ford Algorithm:
Distance from 0 to 0 = 0
Distance from 0 to 1 = 4
Distance from 0 to 2 = 2
Distance from 0 to 3 = 9
```

```
Floyd-Warshall Algorithm:
0  3  2  8
3  0  1  5
2  1  0  6
8  5  6  0
```

Result :

- **Dijkstra's Algorithm** efficiently finds shortest paths when all edge weights are non-negative.
- **Bellman-Ford Algorithm** correctly handles graphs with negative-weight edges and can detect negative cycles.
- **Floyd-Warshall Algorithm** computes shortest paths between every pair of vertices and is useful for complete routing analysis.
- For logistics and transportation networks, Dijkstra is fastest for single-source routing, Bellman-Ford is safer when negative weights exist, and Floyd-Warshall is best for all-route planning.

Time Complexity:

- Dijkstra's Algorithm $\rightarrow O(V^2)$ (or $O((V + E) \log V)$ with a priority queue)
- Bellman-Ford Algorithm $\rightarrow O(V \times E)$
- Floyd-Warshall Algorithm $\rightarrow O(V^3)$

Space Complexity: $O(n)$

GITHUB LINK <https://github.com/Himavarshini-07/AgroChain-Smart-Agriculture-Supply-Chain-Management-System>

Student Signature	Faculty Signature